

Maritime CargoService: Sprint 0

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

June 18, 2026

1. Introduction

Una compagnia di trasporto marittimo di container (d'ora in poi *la committente*) intende automatizzare le operazioni di carico dei container nella stiva della nave (d'ora in poi **hold**). A tal fine prevede di impiegare un robot a guida differenziale (Differential Drive Robot, d'ora in poi **cargorobot**).

L'obiettivo dello Sprint 0 è formalizzare i requisiti forniti dalla committente in modo preciso e non ambiguo, costruire un primo modello logico dei macro-componenti del sistema, evidenziare il *core business*, motivare la scelta del linguaggio di modellazione e definire un primo insieme di piani di test funzionali.

Ogni scelta è strettamente ancorata ai requisiti: il modello qui presentato serve a catturare il comportamento richiesto, senza anticipare decisioni progettuali o scelte implementative che verranno affrontate negli sprint successivi.

La traccia del progetto può essere scaricata dal seguente link^o

2. Requirements

2.1. Requisiti funzionali

- Il servizio da realizzare è **cargoservice**: il customer gli invia una richiesta di carico tramite il pushbutton dell'IOPort.
- L'IOPort è un dispositivo di ingresso/uscita dotato di: pushbutton (richieste di carico), display (stato e messaggi) e sonar (rilevazione presenza container).
- Se l'IOPort è occupata da un container o il sistema è *Out of service*, cargoservice risponde **retrylater**.
- Se tutti gli slot1–slot4 (ciascuno capace di contenere un container) sono occupati, cargoservice **rifiuta** la richiesta.
- Altrimenti, cargoservice entra in stato *engaged*, riserva uno slot libero (senza criteri di ottimizzazione) e restituisce al customer il nome dello slot riservato.
- Fintanto che il sistema è *engaged*, un **LED lampeggia**.
- Dopo l'accettazione, il customer ha un tempo prefissato (es. 30 s) per depositare il container nell'area del sonar.
- Se il customer non deposita entro il timeout, il sistema torna *disengaged*.
- Quando la presenza del container è confermata dal sonar, cargoservice comanda al cargorobot di spostare il container **dall'IOPort** allo **slot5** (adibito al deposito temporaneo di un container prima della collocazione definitiva).
- Il marker device etichetta il container in slot5 e **segnala il completamento**.
- cargoservice comanda al cargorobot di spostare il container **da slot5 allo slot riservato**.
- Il display mostra in ogni momento lo **stato corrente della hold** e il messaggio **"Service working"** durante il normale funzionamento.
- Se il sonar misura $D > D_{\text{FREE}}$ per ≥ 3 s, il sistema passa *Out of service* e mostra **"Out of service"** sul display.
- Il sonar rileva la presenza di un container quando misura $D < D_{\text{FREE}}/2$ per ≥ 3 s.

2.2. Requisiti non funzionali

- La hold è un'area rettangolare piatta con slot1–slot4, slot5 e IOPort.
- Il cargorobot è l'entità software/robotica che dovrà governare un DDR; i requisiti non stabiliscono ancora quale parte sia già disponibile e quale debba essere realizzata.
- Il tempo massimo di attesa per il deposito e la soglia di rilevazione sonar sono parametri prefissati (es. 30 s e 3 s rispettivamente).

2.3. Domande aperte alla committente

Domanda al committente.

Posizione dell'IOPort nella mappa. I requisiti indicano che il cargorobot sposta il container *dall'IOPort a slot5*, lasciando intendere che l'IOPort sia una cella praticabile. Come si colloca nella griglia?

Domanda al committente.

Area di copertura del sonar. La *sensor area* coincide con l'area dell'IOPort o è una cella adiacente? Il chiarimento è essenziale per definire il comportamento di rilevazione e movimentazione.

Domanda al committente.

Richieste concorrenti. Il sistema deve bufferizzare più richieste di carico contemporanee, oppure una nuova richiesta ricevuta mentre il sistema è *engaged* viene semplicemente rifiutata / respinta con *retrylater* senza accodamento? Dai requisiti si interpreta che le richieste **non siano bufferizzate**: se il sistema è occupato, la risposta è immediata (*retrylater* o *refused*) senza code di attesa.

3. Requirement analysis

3.1. Core business

Il **core business** del sistema è la gestione del ciclo di carico di un container. La responsabilità della sequenza applicativa resta in **cargoservice**: il cargorobot è coinvolto per eseguire spostamenti richiesti dal servizio, non per decidere se una richiesta debba essere accettata, rifiutata o sospesa. La sequenza principale ricavata dai requisiti è:

1. Il customer preme il pushbutton dell'IOPort.
2. cargoservice verifica le precondizioni (stato sistema, IOPort, disponibilità slot).
3. Se soddisfatte: stato *engaged*, prenotazione slot, notifica al customer.
4. Il customer deposita il container nell'area del sonar entro il timeout.
5. cargoservice comanda al cargorobot di spostare il container da IOPort a slot5.
6. Il marker device etichetta il container e segnala il completamento.
7. cargoservice comanda al cargorobot di spostare il container da slot5 allo slot riservato; il sistema torna *disengaged*.

3.2. Macro-componenti e natura software

COMPONENTE	RUOLO	STATO
cargoservice	Orchestratore principale. Natura reattiva (richieste in arrivo) e proattiva (comandi al cargorobot, aggiornamenti display). Non può restare privo di controllo: da requisito deve rispondere e agire autonomamente.	Da sviluppare
cargorobot	Entità che governa il DDR per la movimentazione fisica richiesta dal cargoservice. La scelta POJO vs. microservizio è rimandata all'analisi: se fosse un POJO, il cargoservice gli cederebbe il controllo (trasferimento di controllo sincrono) e non potrebbe reagire ad altri eventi durante la movimentazione. Se invece fosse un microservizio, la comunicazione sarebbe asincrona e message-driven, con un disaccoppiamento molto maggiore. Forti motivazioni spingono verso la seconda opzione, ma la decisione è oggetto degli sprint successivi.	Da chiarire / da sviluppare
IOPort	Interfaccia con il customer (push-button + display). Se realizzata come microservizio separato (es. web GUI), le responsabilità sono completamente disaccoppiate da cargoservice e i due possono essere sviluppati in parallelo; l'unica interfaccia condivisa è il messaggio <i>load_request</i> definito in questo sprint.	Fisico fornito sw da sviluppare
sonar	Sensore di distanza. Fisico fornito; driver sw da sviluppare.	Fisico fornito sw da sviluppare
marker device	Etichettatura in slot5. Fisico fornito; sw da sviluppare.	Fisico fornito sw da sviluppare

LED	Indicatore stato <i>engaged</i> . Può essere fisico o virtuale (es. elemento grafico nella web GUI). La scelta dipende dall'infrastruttura disponibile.	Fisico o virtuale sw da sviluppare
hold	Struttura dati per lo stato della stiva (occupazione slot). Passivo.	Da sviluppare

3.3. Motivazione dell'uso del linguaggio QAK

QAK (*Quasi-Actor-based Kotlin*) è il linguaggio messo a disposizione dalla nostra software house (unibo.issLab) per modellare sistemi software distribuiti. Non è quindi assunto come vincolo esterno, ma è lo strumento di modellazione interno che la nostra organizzazione adotta sistematicamente prima di qualsiasi scelta implementativa.

QAK è un DSL basato su Kotlin che permette di definire:

- **Contesti** (`context`) – processi JVM indipendenti che ospitano uno o più attori;
- **Attori** (`qactor`) – entità autonome con macchina a stati esplicita, che comunicano esclusivamente via messaggi;
- **Messaggi** tipizzati (`Request` / `Reply` , `Dispatch` , `Event`) – il vocabolario del dominio reso formale e verificabile.

Dal modello QAK viene generato automaticamente codice Kotlin eseguibile, il che consente di disporre di un **primo prototipo osservabile già nello Sprint 0**, prima ancora di scrivere una riga di logica applicativa.

La scelta è motivata da tre evidenze ricavate direttamente dai requisiti:

- **Natura reattiva e proattiva di cargoservice**: il servizio risponde a stimoli esterni (richieste, segnalazioni sonar e marker device) e avvia autonomamente sequenze di azioni (comandi al cargorobot, aggiornamenti display). Un POJO (Plain Old Java Object), componente passivo attivato da chiamate sincrone, non cattura questo comportamento; un attore QAK sì.
- **Sistema event-driven**: sonar, marker device, IOPort e cargorobot producono o consumano informazioni che non sono naturalmente descrivibili come una singola chiamata sincrona. QAK rende esplicito il vocabolario dei messaggi del dominio, distinguendo richieste con risposta da segnalazioni unidirezionali.
- **Riduzione dell'abstraction gap**: il linguaggio QAK permette di rappresentare le entità rilevanti come attori autonomi e message-driven, mantenendo vicine le frasi dei requisiti e la loro formalizzazione. Il modello è eseguibile e costituisce la base dei test funzionali definiti in questo sprint.

3.4. Formalizzazione dei messaggi QAK

La seguente formalizzazione non introduce ancora scelte di progetto: elenca i messaggi necessari per esprimere i requisiti. **Request/Reply** viene usato quando il requisito prevede una risposta osservabile; **Dispatch** quando il requisito parla di una segnalazione o di un aggiornamento senza risposta diretta.

3.4.1. customer / IOPort -> cargoservice

```
Request  load_request    : loadRequest(none)
Reply    load_accepted   : loadAccepted(slotID)    for load_request
Reply    load_retrylater : loadRetryLater(none)     for load_request
Reply    load_refused    : loadRefused(none)        for load_request
```

3.4.2. sonar -> cargoservice

```
Dispatch sonar_data      : sonarData(distance)
Dispatch container_detected : containerDetected(none)
Dispatch sonar_failure    : sonarFailure(none)
```

Nota: La scelta *Dispatch* (e non *Request*) per i messaggi del sonar è una **ipotesi di analisi**, non una scelta di progetto: il sonar segnala eventi al cargoservice senza aspettarsi una risposta diretta. Se l'analisi rivelasse la necessità di una conferma (es. handshake di rilevazione), il tipo del messaggio andrà rivisto nello sprint 1.

3.4.3. cargoservice -> cargorobot

```
Request  move_to_slot5 : moveToSlot5(none)
Reply    move_done     : moveDone(none)      for move_to_slot5

Request  move_to_slot   : moveToSlot(slotID)
Reply    move_done     : moveDone(none)      for move_to_slot
```

3.4.4. marker device -> cargoservice

```
Dispatch marking_done : markingDone(containerID)
```

3.4.5. cargoservice -> display / LED

```
Dispatch display_hold_state : holdState(state)
Dispatch display_status     : displayStatus(message)
Dispatch led_on             : ledOn(none)
Dispatch led_off            : ledOff(none)
```

3.5. Contesti logici

CONTESTO	COMPONENTI E RESPONSABILITÀ
----------	-----------------------------

ctxCargoService	Contiene l'attore cargoservice. Nucleo del comportamento richiesto e punto di orchestrazione del ciclo di carico.
ctxIO	Raggruppa le entità legate all'IOPort e ai dispositivi citati dai requisiti: ioport, sonar, led, markerdevice.
ctxRobot	Raggruppa le entità legate al cargorobot e alla movimentazione richiesta.

3.6. Schema della hold

Legenda: H = HOME S = sonar/sensor area 1-4 = slot1-4
5 = slot5 I = IOPort X = ostacolo . = libero

	col0	col1	col2	col3	col4	col5	col6
riga0	[H]	[S]	[.]	[.]	[.]	[.]	[.]
riga1	[.]	[1]	[X]	[X]	[2]	[.]	[.]
riga2	[.]	[.]	[.]	[.]	[X]	[5]	[.]
riga3	[.]	[3]	[X]	[X]	[4]	[.]	[.]
riga4	[.]	[.]	[.]	[.]	[.]	[.]	[.]
riga5	[I]	[X]	[X]	[X]	[X]	[X]	[X]

Nota: La posizione esatta dell'IOPort e del sonar richiede conferma dalla committente. Lo schema è una prima interpretazione fedele alla figura allegata ai requisiti.

4. Test plan

I test funzionali verificano il comportamento del sistema rispetto ai requisiti, in modo indipendente dall'implementazione.

4.1. Accettazione della richiesta

Precondizioni: *disengaged*, non *Out of service*, IOPort libera, almeno uno slot libero.

Azioni: il customer preme il pushbutton.

Risultato: risposta *load_accepted* con slot riservato; sistema *engaged*; LED lampeggiante.

4.2. Rifiuto (hold piena)

Precondizioni: *disengaged*, non *Out of service*, IOPort libera, slot1–slot4 tutti occupati.

Azioni: il customer preme il pushbutton.

Risultato: risposta *load_refused*; sistema *disengaged*; LED spento.

4.3. Retrylater (Out of service)

Precondizioni: sistema *Out of service*.

Azioni: il customer preme il pushbutton.

Risultato: risposta *load_retrylater*; stato immutato.

4.4. Retrylater (IOPort occupata)

Precondizioni: *disengaged*, non *Out of service*, IOPort occupata.

Azioni: il customer preme il pushbutton.

Risultato: risposta *load_retrylater*; sistema *disengaged*.

4.5. Timeout deposito container

Precondizioni: sistema *engaged*.

Azioni: il customer non deposita il container entro il tempo prefissato.

Risultato: sistema *disengaged*; LED spento.

4.6. Ciclo completo di carico

Precondizioni: *disengaged*, non *Out of service*, IOPort libera, slot disponibile.

Azioni: (a) pushbutton -> (b) *load_accepted* -> (c) deposito container -> (d) rilevazione sonar -> (e) cargorobot: IOPort->slot5 -> (f) etichettatura -> (g) cargorobot: slot5->slot riservato.

Risultato: container nello slot riservato; display aggiornato con “*Service working*”; sistema *disengaged*; LED spento.

4.7. Malfunzionamento sonar

Precondizioni: sistema operativo.

Azioni: sonar misura $D > D_{FREE}$ per ≥ 3 s consecutivi.

Risultato: sistema *Out of service*; display mostra “*Out of service*”.

4.8. Rilevazione container (sonar)

Precondizioni: sistema *engaged*; sensor area vuota.

Azioni: deposito container; sonar misura $D < D_{FREE}/2$ per ≥ 3 s.

Risultato: cargoservice riceve *container_detected* e avvia la movimentazione.

5. Project

Dall’analisi dei requisiti si propone, come ipotesi iniziale, una struttura a **tre sprint** con integrazione progressiva dei componenti. La suddivisione serve a organizzare il lavoro e i test, non a fissare decisioni architetture definitive.

5.1. Sprint 1: Core business

Goal: realizzare il primo nucleo eseguibile di cargoservice con collaboratori simulati.

Funzionalità: ciclo di carico completo con collaboratori simulati, modello dello stato della hold, LED simulato, timeout.

5.2. Sprint 2: IOPort, display e sonar

Goal: rendere esplicita l’interazione con IOPort, display e sonar a livello software.

Funzionalità: ioport come interfaccia web, sonar simulato (rilevazione container e malfunzionamento), aggiornamento display con stato e messaggi.

5.3. Sprint 3: Dispositivi fisici

Goal: integrare, dove disponibili, i dispositivi fisici o i servizi forniti e verificare il comportamento end-to-end.

Funzionalità: cargorobot e servizio DDR disponibile, sonar fisico, marker device fisico, LED fisico.

6. Team di lavoro

NOME E COGNOME	RUOLO
Davide Chirichella	Membro del gruppo
Gabriele Doti	Membro del gruppo
Daniele Maccagnan	Membro del gruppo